Memorable: Compact Verbatim Memory for Long Conversations

Nikhil Krishnaswamy[1,2], Advaiyt Sane[2,3]

[1]Stanford University, USA; [2]Memorable, San Francisco, USA; [3]Carnegie Mellon University, USA

Correspondence nikhil@memorable.sh, advaiyt@memorable.sh
Code, results and registered predictions github.com/MemorableOrg/compact-verbatim-memory

Abstract

Assistants now hold conversations that run for months, yet a frozen language model can use that history only through its context window, so a memory system has to select the few turns that answer each question. Mem0, Zep and similar systems build their memory with a language model that reads every exchange and extracts facts. Each exchange therefore costs a model call to store, and the stored fact is the model's paraphrase of the user's words. We present Memorable, a memory that stores the conversation itself and indexes it with fixed code and a local embedder. It keeps every turn, strips formatting and filler with fixed rules, and stores the text with zlib together with an 8-bit embedding of each turn. The first BEAM-1M conversation takes 1.47 MB and about 30 seconds to build on a laptop CPU. For each question, a small planner proposes two to four search queries, keyword and embedding rankings are fused and rescored by a local reranker, and a frozen reader answers from 20 or 60 turns. On BEAM-1M, 700 questions over 35 conversations scored with one judge for both systems, Memorable passes 67.3% of questions and Mem0 63.9%; the registered 95% interval on the gap includes zero, so we report a tie. On the ten 10M-token conversations Mem0 published answers for, run once after registering predictions, Memorable passes 56.0% against 46.5% and is ahead in every conversation. Because Memorable keeps both versions of a fact the user changed, it resolves contradiction questions at 58.6% against Mem0's 7.1%. Building Mem0's memory for one 1M conversation costs about 855 model calls, $7.50 and 5.4 hours and takes 80 MB on disk, extrapolated from the 80% we ran. Memorable reads more tokens per question, so Mem0 is cheaper in total after about 460 questions on a conversation. On LongMemEval-S our search places every gold session in the top 10 for 95.5% of the 375 test questions, where the two strongest published systems report 93.8% and 94.5%. Code, per-question results and the registered predictions are released with the paper.

1 Introduction

An assistant that someone uses for months should remember what they said in week one. A language model can use that history only through its context, and a year of conversation does not fit, so a memory system has to select, for each question, the small part of the history that answers it [8, 9]. The selection is hard for three reasons: the history keeps growing, the questions range from a single fact to the order of every event on a topic, and the memory is built once, for every question that comes later.

Most memory systems today write their memory with a model. Mem0, Zep and A-MEM pass every exchange through a language model that writes facts, summaries or graph edges, and store those in place of

the conversation [3, 10, 16]. This has three costs. The model is paid once per exchange, so the cost of the memory grows with the length of the conversation rather than with its use. Two builds of the same history can differ, because the writer samples at any temperature above zero. And when the writer replaces a fact with its newer version, the earlier statement is gone from the memory, and with it the answer to "did I ever say otherwise?" In Mem0's open-source release an UPDATE overwrites the earlier fact; its hosted pipeline is closed. On BEAM's contradiction questions, which ask exactly that, Mem0's published answers pass 7.1% under the one judge (Figure 3). A second family, latent memories such as Memorilla and xRAG, encodes the history into vectors that the model reads directly [2, 6]. These need training against each base model and access to its embedding layer, and hosted models do not expose that layer. Plain retrieval over the raw text keeps everything and is free to build, and it is where we start. Its weakness is that one query finds one session: on questions whose evidence is spread over several sessions, our retrieval-only run trails Mem0 by 20.0 points (Section 4.1).

We present Memorable, a memory for long conversations built by fixed code from the turns themselves (Figure 1). The write path keeps every turn with its speaker and date. Fixed rules remove markdown decoration, filler such as "Certainly!", and assistant sentences repeated word for word earlier in the conversation. The text is stored with zlib beside each turn's embedding from a 33M-parameter local model, quantised to one signed byte per dimension. Across the 35 BEAM-1M conversations the memory averages 1.77 MB against 7.84 MB for the raw turns and float vectors, and all 35 build in under 16 minutes on a laptop. The search path runs once per question. A planner call names the question type and proposes two to four queries; keyword and embedding rankings for each query are fused, each turn lends part of its score to its partner in the user-assistant exchange, and a local cross-encoder rescores the top 100. The reader always sees the user's standing instructions, takes at most six turns from any one session, and reads 20 turns in conversation order, or 60 when the question asks for an order, a summary or an aggregate. A frozen reader then answers under fixed rules.

We evaluate on BEAM's 1M-token tier, 700 questions over 35 conversations and ten abilities, scoring our answers and Mem0's published answers with one judge. Memorable passes 67.3% and Mem0 63.9%. The registered 95% interval on the 3.4-point gap runs from −0.4 to +7.3 and includes zero, so we report a tie; the paired interval, +0.3 to +6.6, excludes zero. Base retrieval, the search path before the planner, compaction and 8-bit vectors are added, already ties Mem0 at 64.7%, and the planner and the two list rules close most of the gap on multi-session questions (Section 4.1). On the 10M-token tier, 200 questions over ten conversations run once after registering predictions, Memorable passes 56.0% and Mem0 46.5%, a gap whose interval excludes zero. Building Mem0's memory for one 1M conversation with its open-source release costs about 855 extraction calls, $7.50 and 5.4 hours and takes 80 MB on disk, extrapolated from the 80% we ran; Memorable builds the same conversation in about 30 seconds of CPU and stores 1.47 MB (Section 4.2). On LongMemEval-S our search places every gold session in the top 10 for 95.5% of test questions (Section 4.3). We registered a numbered prediction before every run and report which held (Appendix A).

The paper makes four contributions: (1) a write path of fixed code, three compaction rules, zlib storage and 8-bit vectors, that stores the compacted text and one vector for every turn of a conversation; (2) a search path of a small planner, hybrid retrieval with a pair rule, a local cross-encoder and two rules on the retrieved list, which is where the gain on multi-session questions comes from; (3) a comparison with Mem0 on BEAM-1M and BEAM-10M under one judge for both systems, with a prediction registered before every run and a report of whether it held; and (4) a cost model with measured build and read costs that gives the number of questions at which a model-written memory becomes cheaper, released together with the code, the per-question results and the registered predictions.

2 Memorable

A conversation is a sequence of turns $T = (t_1, \ldots, t_n)$, each with a speaker, a date and its text x_i. Memorable stores T once with fixed code and a local embedder. Its model calls happen at question time (Figure 1): a small planner that proposes search queries for the question q and a frozen reader that receives k turns in conversation order and answers. Everything between the two is fixed code, a 33M-parameter embedder and a 278M-parameter cross-encoder, all run on a laptop CPU.

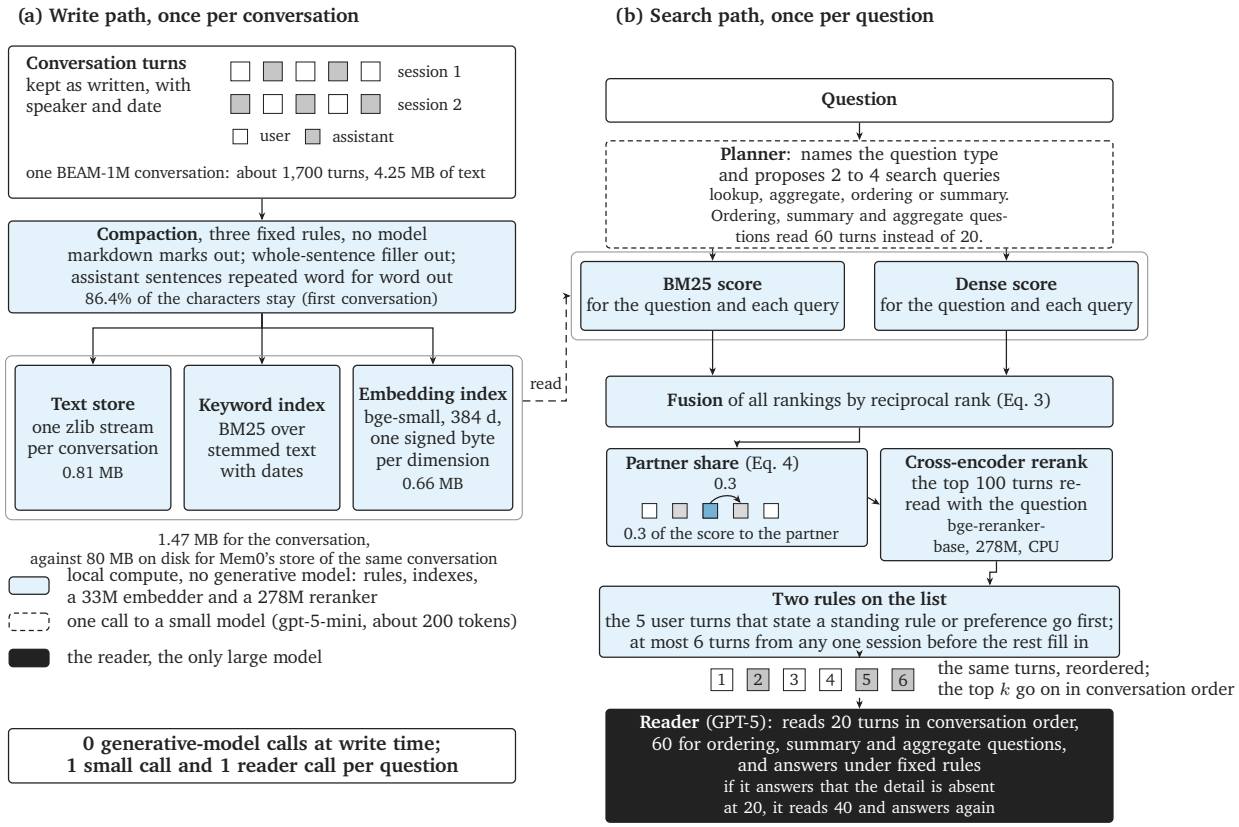


Figure 1: Memorable. (a) The write path runs once per conversation on fixed code and a local embedder. Turns are compacted by three rules, the text is stored with zlib and each turn's embedding as 8-bit integers. (b) The search path runs once per question. A planner proposes two to four queries; keyword and embedding rankings are fused; each turn lends a share of its score to its partner; a cross-encoder rescores the top 100; standing instructions are placed first; at most six turns come from one session; the reader answers from 20 or 60 turns.

2.1 Store

Compaction. Assistant prose in a long conversation carries a great deal of markdown decoration, pleasantries and caveats repeated hundreds of times, and each of them costs reader tokens when its turn is retrieved. We remove them with three deterministic maps applied to every turn, $C = R \circ F \circ M$. M strips markdown decoration (heading marks, emphasis stars, horizontal rules, table separator rows and code-fence lines) and replaces table pipes with a spaced bar so that cell contents stay readable. F removes a sentence from either speaker when the whole sentence is a pleasantry or a sign-off, such as "Certainly!" or "Let me know if you need further assistance!". R removes an assistant sentence longer than 40 characters whose lower-cased, punctuation-stripped form already appeared earlier in the conversation, keeping the first copy. All three maps are string operations. On the first BEAM-1M conversation (1,710 turns, 4.25M characters) the three maps keep 93.1%, 91.2% and 86.4% of the characters in turn, and over all 35 conversations 83% remain (Figure 6). On the 500K development set compaction moved the pass rate by less than two points while cutting the tokens sent to the reader from 23.7K to 19.2K per question.

Storage. The compacted turns, with their ids, speakers, session indices and dates, are written as one zlib stream per conversation; the session index adds 363 bytes per 1M conversation on average. Each turn's first 256 tokens are embedded once with bge-small-en-v1.5 [14], $d = 384$, the vector is unit-normalised, and one signed byte is stored per dimension,

$$\mathrm{q}_i = \mathrm{round}(127\,\mathrm{e}_i), \qquad \hat{s}_i = \mathrm{e}_q^{\top}\mathrm{q}_i/127, \tag{1}$$

so that 384 bytes replace 1,536. The score error is at most $|s_i - \hat{s}_i| \le \|e_q\|_1/254 \le \sqrt{d}/254 \approx 0.077$. We can afford this because the cross-encoder of Section 2.2 rescores the top 100 candidates from their text, so the error affects which turns reach that list and leaves their order within it to the cross-encoder; Appendix F compares recall with float and 8-bit vectors. The bytes stored for one conversation are

$$B(T) = |\mathrm{zlib}(C(T))| + n\,d. \tag{2}$$

Over the 35 BEAM-1M conversations, 159.8 MB of text and 114.6 MB of float vectors become 33.5 MB and 28.7 MB: 62.1 MB stored against 274.4 MB raw, or 1.77 MB per conversation. A stemmed BM25 index over the compacted text [11], with each turn's date in its document, is rebuilt from the stored text when a conversation is loaded, so it is left out of $B(T)$.

2.2 Search

Planner. Which turns answer a question depends on its type. A lookup needs the one turn that states a fact, whereas an aggregate, an ordering or a summary needs evidence from many sessions, and a single query rarely reaches them all (Section 4.1). One call to a small model (gpt-5-mini, about 200 input tokens, about $0.001 per question) therefore labels the question as a lookup, an aggregate, an ordering ("order" in the prompt) or a summary and proposes $m \le 4$ search queries $q_1, \ldots, q_m$ that together cover it. A question about the days, weeks or months between two events is labelled a lookup, so the reader receives 20 turns; aggregates receive 60.

Fusion. We rank the turns for the question and for each planner query twice, by BM25 and by embedding cosine, so that a turn can be found by its words or by its meaning. When a turn's date matches a date named in the question, half of the highest BM25 score in that list is added to the turn's BM25 score, which pulls the turns of a named day to the front. Each list is cut to its top 200 and all $2(m+1)$ lists are fused by reciprocal rank [4],

$$s_i = \sum_{\ell} \frac{1}{60 + r_\ell(i)}, \tag{3}$$

where $r_\ell(i)$ is the rank of turn i in list ℓ. Each turn then lends 0.3 of its score to its neighbour $p(i)$, the other turn of its user-assistant exchange,

$$s'_i = s_i + 0.3\, s_{p(i)}, \tag{4}$$

because the question often matches one half of the exchange while the answer sits in the other. A local cross-encoder, bge-reranker-base [1], reads the question beside the first 1,500 characters of each of the 100 turns with the highest s' and scores how well the turn answers it. The cross-encoder ranking is fused with the earlier ranking by Equation 3 once more, and the 100 candidates stay above every other turn.

Standing rules and a session cap. Two rules act on the fused list before the reader sees it. The first keeps the user's standing instructions in view. User turns that open with "always" or "never", or whose first 400 characters contain "from now on", "I prefer" or "my preference", form a pool of standing rules, and the five closest to the question by embedding are placed first, so a rule stated early in the conversation reaches the reader on every later question. The second spreads the read set across sessions: turns are taken from the top of the list, a turn is skipped once six turns from its session have been taken, and the skipped turns fill whatever places remain. The reader receives the first $k = 20$ turns in conversation order, or $k = 60$ for ordering, summary and aggregate questions. If a reader given 20 turns answers that the detail is absent, it is given 40 turns and answers again.

2.3 Reader

The reader (GPT-5) sees each turn, cut to 6,000 characters, with its date and speaker, and answers under fixed rules (Appendix D). The rules about evidence tell it to use only the excerpts, to keep exact names, numbers and dates, to follow standing instructions, and to say that the conversation lacks a detail rather than fill one in. The rules about answers cover the cases a verbatim memory raises that a model-written one has already

resolved. The reader prefers the most recent of two conflicting excerpts and says that the fact changed, lists instances with their dates before giving a count, merges repeated mentions of one event, and gives only the most recent value when the question asks for the current one. Four more rules of the same kind, on repeated instructions, relative dates, summaries and the time between two events, are in the full text.

2.4 Cost

A memory that a model writes costs c_w per exchange, N in all, and c_r per question, Q in all. Memorable has a write cost of zero and costs $c_p + c_r'$ per question, with c_p the planner call and c_r' the reader:

$$\mathrm{Cost}_{\mathrm{writer}}(Q) = Nc_w + Qc_r, \qquad \mathrm{Cost}_{\mathrm{ours}}(Q) = Q\,(c_p + c_r'), \qquad Q^* = \frac{Nc_w}{c_p + c_r' - c_r}. \tag{5}$$

With the measured values of Section 4.2, $Nc_w = \$7.50$ and $c_p + c_r' = \$0.047$, and an estimated $c_r = \$0.0307$ for Mem0's 200 retrieved facts, the break-even is $Q^* \approx 460$ questions per conversation (Figure 7). Below it Memorable is cheaper in total; above it Mem0 is, because its reader sees fewer tokens per question. Storage is outside the equation and favours Memorable at every Q.

3 Experiments

BEAM. BEAM has conversations of 100K to 10M tokens, each with 20 questions spread over ten abilities [12]. Our main tier is 1M: 35 conversations and 700 questions, 70 per ability. We chose the settings of Section 2 on 200-question development sets from the 100K and 500K tiers and registered a prediction before each run. The abilities we chose to target came from an earlier 1M comparison of an earlier version of Memorable, so the 1M tier informed the design. Mem0's answers come from the per-question file it published for its hosted Platform v3 pipeline, a run dated 2026-03-26 with GPT-5 reading its top 200 memories [7]. It is the only per-question BEAM file Mem0 publishes for the 1M tier, and Mem0 publishes the same for all ten conversations of the 10M tier, 200 questions.

Runs. On the 1M tier we scored one run of each of two configurations, both with GPT-5 as reader. Base retrieval runs the fusion, the pair rule, the cross-encoder and the standing pool over the raw turns with float vectors. The full method adds the planner, compaction and 8-bit vectors. Base retrieval ran first.

After base retrieval scored, and before the full run, we wrote a stopping rule: run the full method only if base retrieval plus half of the development gain reached Mem0 plus 4 points, 67.9. With the 500K gain the forecast was 69.3 and the rule passed. We launched the full run and stopped it during indexing, ahead of the first answer, because a check on the 100K tier showed a regression on temporal reasoning. We then changed the reader rules once, for temporal questions, with base retrieval's 1M scores by ability known at the time. After the change the pooled development gain gave a forecast of 67.5, 0.4 short of the rule. We ran the full method anyway. That run paused once after 42 questions and resumed with more workers (Appendix A).

On the 10M tier we ran the full method once, with the 1M settings, after registering predictions for both systems. Mem0's own-judge scores on that tier were public when we registered, so our prediction for Mem0 was not independent of its score. A first launch crashed after one question because the 10M file stores turn ids as text; we fixed the loader, discarded that answer and scored only the full run.

One judge. BEAM scores each answer against a rubric of items with a model judge, and Mem0 scored its file with a different judge from the one in BEAM's harness. We therefore re-score both systems' answers with one model, Claude Sonnet 5.5, under BEAM's rubric prompt. Both systems read and judge with other models (Appendix C names them), and we fixed the choice before scoring any answer. An answer passes when its mean item score is at least 0.5, the threshold Mem0 reports. All BEAM numbers in the body use this judge; Appendix C gives each system's own-judge numbers. Intervals on the gap are 95% bootstraps over the tier's conversations, 35 on 1M and ten on 10M, with 2,000 draws. We registered the independent interval, which resamples conversations separately for each system. We also report the paired interval, which resamples the same conversations on both sides. The paired form is the usual choice when both systems answer the same

conversations, and it is narrower. We call a gap whose interval includes zero a tie. A judge output that lacks a score counts as 0; we logged these only from the 10M run onward (Appendix C).

Build cost. We built Mem0's memory for the first 1M conversation with its open-source release, mem0ai 2.0.0, using gpt-5-mini as extractor and our bge-small embedder, and metered every model call, token and byte. Its extraction prompt and per-batch extraction call are unchanged through the current 2.2.1 of September 2026. A $6 cap stopped the build at 80% of the conversation, and we extrapolate the figures to the whole. The 80 MB is Mem0's vector database on disk, index included; our 1.47 MB counts the stored text and vectors.

LongMemEval-S. LongMemEval-S has 500 questions over histories of about 50 sessions [13]. We report retrieval only, as all-gold recall, the share of questions whose gold sessions all appear in the top k. Of the 470 questions with gold labels, 95 fall in the development split, on which we made the retrieval choices (100 question ids chosen by hash), and 375 in the test split; we report the test split first and the others beside it. EdgeMem and MemGAS report 93.8% and 94.5% on a metric we read as the same [5, 15].

4 Results

The comparison is scoped to the two BEAM tiers for which Mem0 publishes per-question answers, scored by one judge for both systems, and to retrieval alone on LongMemEval-S.

4.1 BEAM

Under one judge, base retrieval passes 64.7% of the 700 questions and Mem0 63.9%, a gap of $+0.9$ with interval $[-3.0, +4.4]$: a tie (Figure 2). The full method passes 67.3%, 3.4 points above Mem0, with interval $[-0.4, +7.3]$. That interval also includes zero, so we report a tie here too (Figure 3; Table 2 has the numbers). Resampling the same conversations on both sides gives a paired interval of $[+0.3, +6.6]$; we registered the independent form and keep the tie.

Contradiction questions. The largest gap between the two systems is here. These questions ask about a fact the user changed, and the rubric asks the answer to name both statements. In the first conversation the user writes "I've completed the translation microservice and it's working great" and, 158 turns later, that they have never completed it. Our reader sees both turns and says so: under the one judge the full method passes 58.6% of these questions and base retrieval 65.7%, against 7.1% for Mem0. The gap holds under both judges that scored both systems, and its size varies with the judge. Mem0's own judge scores the same answers 48.6% and BEAM's judge 15.7%, and under BEAM's judge base retrieval scores 64.3% and the full method 42.9% (Appendix C). In Mem0's open-source release an UPDATE replaces the earlier fact, so the memory holds one version. Its hosted pipeline is closed, so we cannot confirm that the same step produced the 7.1%.

What search adds. Base retrieval trails Mem0 on the abilities whose evidence is spread over several sessions: multi-session reasoning by 20.0 points, event ordering by 17.1 and summarization by 7.1. One query finds one session, whereas a memory that a model wrote holds facts from every session in one store. The planner, the 60-turn read and the session cap are aimed at this gap. On the development sets (400 questions pooled over the 100K and 500K tiers, gpt-5-mini reader, BEAM's judge) they raise the pass rate from 74.2% to 79.8%, with multi-session reasoning moving from 67.5 to 82.5, event ordering from 52.5 to 70.0 and summarization from 65.0 to 80.0 (Figure 4). On the 1M tier under one judge the same components move multi-session reasoning from 55.7 to 70.0, event ordering from 34.3 to 54.3 and summarization from 50.0 to 54.3. They cost ground on two abilities: knowledge update falls from 62.9 to 52.9 and contradiction resolution from 65.7 to 58.6, and under BEAM's judge the contradiction drop is larger, 64.3 to 42.9. Of the 18 contradiction questions lost under that judge, half dropped a gold turn from the read set; in the other half the reader listed both statements as a dated timeline and left their conflict unnamed. The rubric requires the conflict to be named (Appendix C).

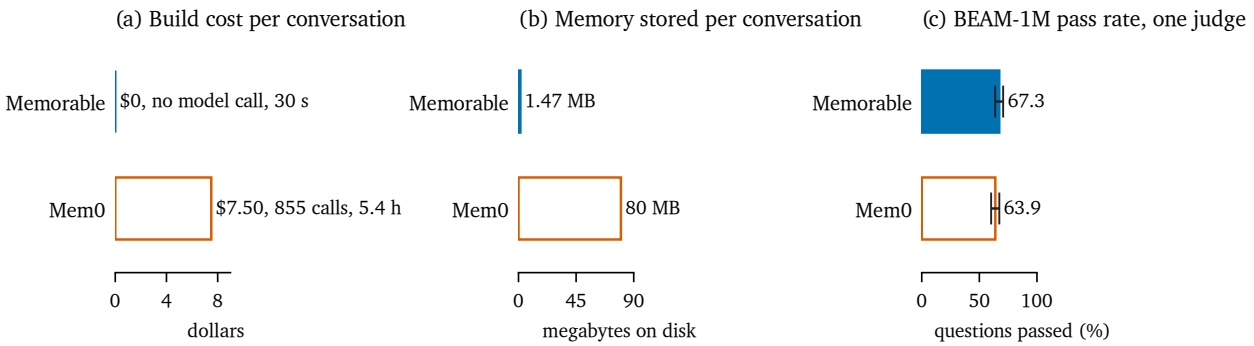


Figure 2: Build cost, bytes stored and pass rate for one 1M-token BEAM conversation. Left: model cost, calls and time to build the memory, measured on Mem0's open-source release over 80% of the conversation and extrapolated to the whole. Middle: bytes stored on disk. Right: pass rate on BEAM-1M under one judge, 700 questions, with 95% Wilson intervals. Per-question cost is in Figure 7.

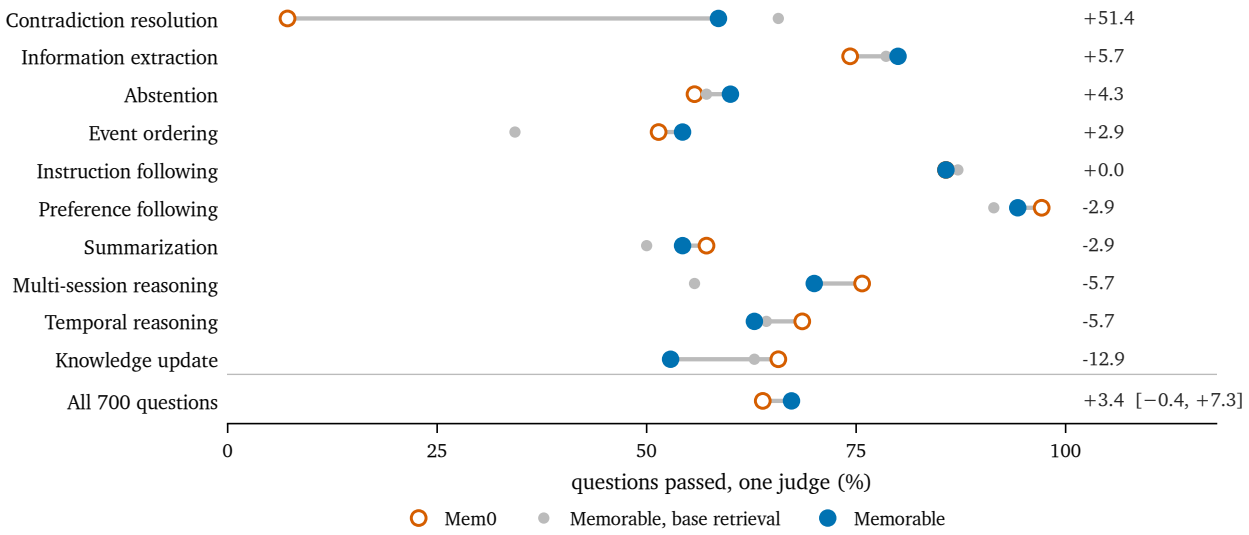


Figure 3: BEAM-1M by ability, one judge, 70 questions per ability. Hollow dots are Mem0, filled blue dots are Memorable and small grey dots are base retrieval. The grey line joins Mem0 to Memorable; the number at the right is Memorable minus Mem0 in points. The last row is all 700 questions with the bootstrap interval on the gap.

Where Mem0 stays ahead. Mem0 leads on knowledge update by 12.8 points, and by 2.9 to 5.7 points, two to four questions of 70, on temporal reasoning, multi-session reasoning, preference following and summarization (Figure 3). Knowledge-update questions ask for the current value of a fact that changed. A memory that overwrites holds only that value, whereas our reader sees both turns and has to pick the later one; we did not test whether this explains the gap.

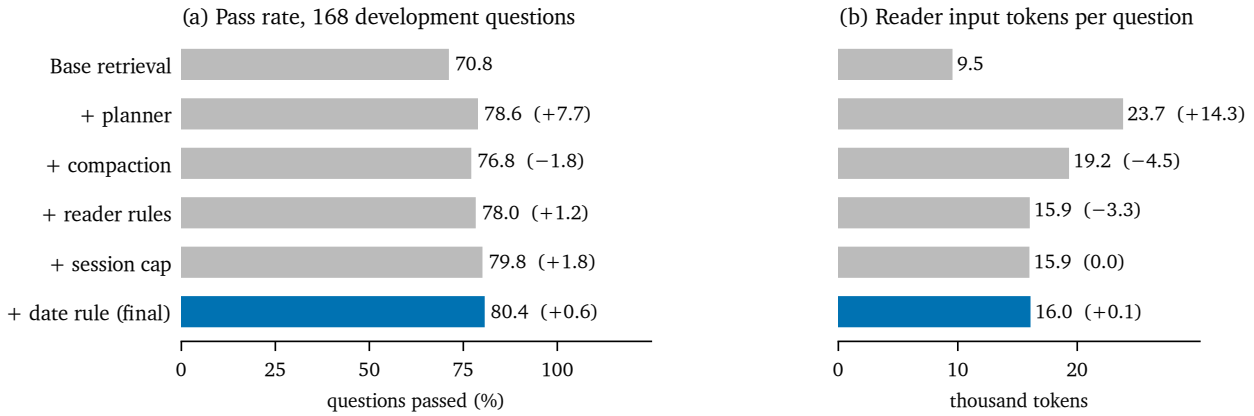


Figure 4: Adding the search components one at a time on the 500K development tier (development questions, separate from the 1M result): 168 questions every run answered, gpt-5-mini reader, BEAM's judge. Left: pass rate. Right: input tokens handed to the reader per question. The planner adds 7.7 points and 14.3K tokens; compaction and the reader rules return 7.8K of those tokens.

The 10M tier. Mem0 publishes answers for the ten conversations of BEAM's 10M-token tier, 200 questions, and we ran the full method on them once after registering predictions (Appendix A). Under the one judge Memorable passes 56.0% and Mem0 46.5%, a gap of 9.5 points with a registered interval of $[+0.5, +18.0]$ and a paired interval of $[+7.0, +12.0]$ (Figure 5). Memorable is ahead in each of the ten conversations, by one to three questions of 20; with all ten differences positive, a two-sided sign-flip test gives $p = 2/1024 = 0.002$. Contradiction resolution is 60.0% against 10.0% and temporal reasoning 50.0% against 20.0%, while Mem0 stays ahead on preference following, 90.0% against 75.0%, and on multi-session reasoning, 30.0% against 25.0%. Each ability has 20 questions here, so one question moves it by 5 points, and temporal reasoning changes sign between tiers (Mem0 ahead by 5.7 points on 1M, Memorable ahead by 30.0 here); we do not interpret that. The reader took 16,172 input tokens per question, and the run cost $10.21, of which $9.49 was the reader and the rest the planner and BEAM's judge in the run loop. The memory of a 10M-token conversation takes 16.7 MB, above the 10 MB we predicted.

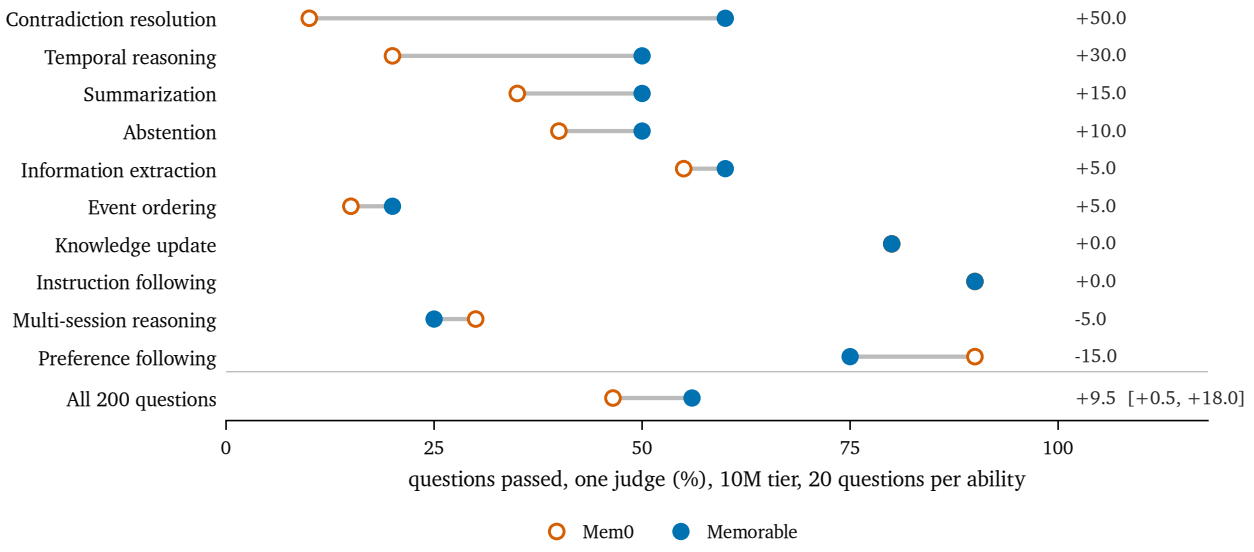


Figure 5: BEAM-10M, 200 questions over ten conversations, one judge. Hollow orange: Mem0 (Platform v3, GPT-5). Filled blue: Memorable, full method with the 1M settings, run once. Right: the gap in points; the interval beside the All 200 questions row is the registered 95% interval.

4.2 Cost and storage

Build. Building the memory of the first 1M conversation with Mem0's open-source release and gpt-5-mini as extractor took 683 extraction calls for 80% of the turns before a $6 cap stopped it: 6.85M input and 2.14M output tokens, 3,788 memory entries, 260 minutes at one worker and 64 MB on disk. Scaled to the whole conversation, that is about 855 calls, $7.50, 5.4 hours and 80 MB (Table 3). Memorable builds the same conversation in about 30 seconds of CPU and stores 1.47 MB, 0.81 MB of compressed text and 0.66 MB of 8-bit vectors (Figure 6). Mem0's facts take less text than our turns: about 4,700 entries for the whole conversation hold 1.15 MB against our 3.67 MB of compacted turns before compression, or 0.34 MB against 0.81 MB after zlib. Each entry also carries a float vector, 2.8 per turn and 7.3 MB in all, and the rest of the 80 MB is its vector database and index.

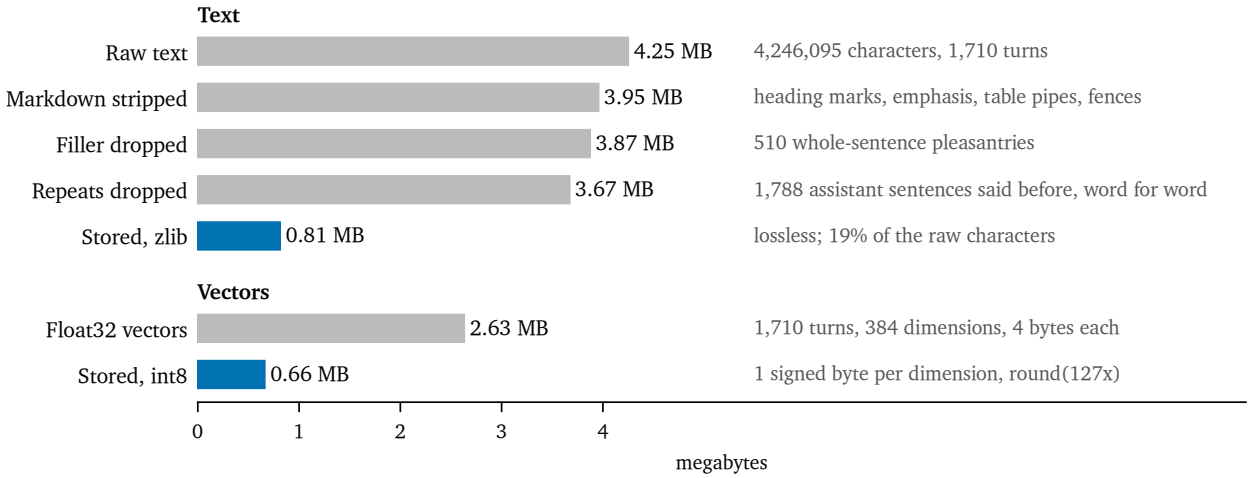


Figure 6: Bytes kept at each step for the first 1M conversation. Text: raw, after each compaction rule, and after zlib. Vectors: float32 and 8-bit.

Per question. Memorable's reader, GPT-5, takes 17,200 input tokens and produces 2,450 output tokens per question, $0.046 at GPT-5 prices measured over the 700 questions, plus about $0.001 for the planner call. Mem0's reader takes 200 facts of about 242 characters, about 12,100 input tokens, plus about 450 for the question and instructions. Mem0 publishes the facts and leaves out token counts, so we assume 1,500 output tokens; GPT-5 prices then give $0.0307, which we write as $0.031. With Equation 5 the two systems cost the same after about 460 questions on one conversation (Figure 7).

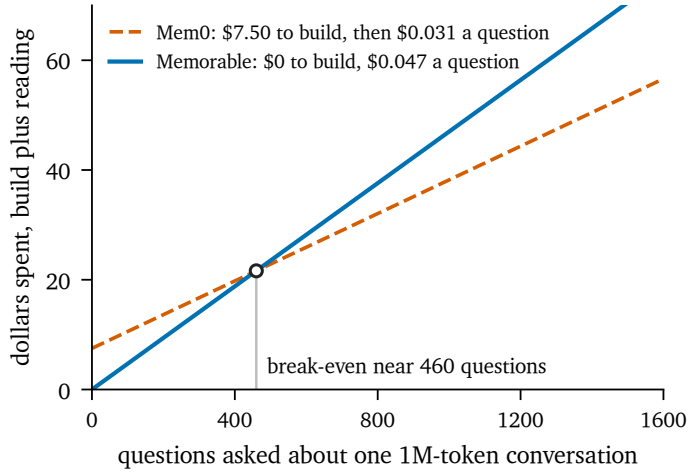


Figure 7: Dollars spent on one 1M-token conversation as questions accumulate. Mem0 starts at its measured build cost; Memorable starts at zero. Mem0's per-question cost is estimated from the size of its retrieved facts; ours is measured.

4.3 LongMemEval-S

Retrieval. Keyword and embedding rankings fused by reciprocal rank and rescored by the cross-encoder place every gold session in the top 10 for 95.5% of the 375 LongMemEval-S test questions with gold labels. EdgeMem reports 93.8% and MemGAS 94.5% [5, 15], and we read the published metrics as the same measure. The 95 development questions, on which we made the retrieval choices, score 92.6%; all 470 together score 94.9%. On LongMemEval-S most gold turns are the user's, so a rule that hands the reader every user turn in the top 60, and an assistant reply only when it ranks in the top 10, brings the input to 6,777 tokens and keeps every gold turn for 97.9% of development questions. We have not run a reader on that setting.

5 Related work

Memories a model writes. MemGPT pages summaries in and out of the context window [8]; Mem0 extracts facts from each exchange and adds, updates or deletes entries in a vector store [3]; Zep builds a temporal knowledge graph [10]; A-MEM links model-written notes [16]. All of them pay a model once per exchange to build the memory and store the model's words in place of the user's.

Latent memories. These replace text with vectors that a model reads directly: xRAG compresses a document to one token [2], and Memorilla trains a memory module that gathers query-relevant information from an encoded collection into 16 tokens [6]. These need training against each base model and access to its embedding layer, whereas Memorable works with a frozen reader behind a hosted API.

Retrieval over the raw turns. Hybrid retrieval with reciprocal rank fusion [4] and cross-encoder reranking [1] are standard in search. LongMemEval's own analysis of memory design found that storing the rounds of a conversation scores higher than storing facts extracted from them on every question type but multi-session reasoning [13]. Our retrieval-only run also trails Mem0 there. Memorable builds on that finding: a hybrid retriever over verbatim turns, plus a planner, two rules on the retrieved list and fixed reader rules, is built by fixed code and ties a memory that a model wrote on BEAM-1M under one judge. We evaluate on BEAM [12] and LongMemEval [13] and registered a prediction before every run (Appendix A).

6 Conclusion

This paper asked whether a memory built by fixed code from the turns themselves can match one that a model writes. Memorable stores the conversation itself, compacted by fixed rules and compressed with zlib, and the first BEAM-1M conversation takes 1.47 MB; its model calls are the planner and the reader, both at question time. On BEAM-1M under one judge it passes 67.3% of questions against Mem0's 63.9%, a tie, since the interval on the gap includes zero; on the 10M tier it passes 56.0% against 46.5% and is ahead in each of the ten conversations. A memory that a model writes holds less text than the conversation, and it holds facts from every session in one store. That is where base retrieval trails. The planner and the two list rules close 14.3 of the 20.0-point gap on multi-session questions, from 55.7 to 70.0 against Mem0's 75.7. Keeping every turn is what lets the reader see both sides of a changed fact, and it is also what costs more tokens per question, so Mem0 is cheaper in total after about 460 questions on a conversation.

Limitations. Several limitations must be acknowledged. (1) The judge is a model. We score both systems with one judge, but a different judge can still move an ability by tens of points: Mem0's contradiction answers score 48.6% under its own judge, 15.7% under BEAM's judge and 7.1% under Claude Sonnet 5.5 (Appendix C). (2) Mem0's answers are from the file it published in March 2026; a fresh run of its hosted service might score differently, and we could not run it ourselves. (3) We tuned the full method on the 100K and 500K tiers and ran it once on 1M and once on the 10M questions Mem0 published, and the contradiction-resolution drop between base retrieval and the full method shows that a development set of 400 questions misses effects of this size. (4) Mem0's per-question read cost is estimated from the size of its retrieved facts; ours is metered. (5) On LongMemEval-S we measured retrieval alone, on the setting that fits Mem0's token

budget. (6) Eight-bit vectors change the share of gold turns found by at most 0.4 points on the development set (Appendix F). (7) On the first 1M conversation we inspected compaction's removals by frequency and found pleasantries and repeated closings; we have not read every removed sentence, so a lost fact would have escaped us. (8) The filler and repeat rules are written for English assistant prose and need new patterns for other languages or for agent tool logs. To address these, future work should score a fresh run of Mem0's hosted pipeline under the same judge, run a reader on the LongMemEval-S setting, and read every removed sentence.

References

[1] Jianlv Chen, Shitao Xiao, Peitian Zhang, et al. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.

[2] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. xRAG: Extreme context compression for retrieval-augmented generation with one token. *arXiv preprint arXiv:2405.13792*, 2024.

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL https://arxiv.org/abs/2504.19413. arXiv:2504.19413.

[4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of SIGIR*, 2009. URL https://doi.org/10.1145/1571941.1572114.

[5] Zeyang Cui, Jiannong Cao, Zhiyuan Wen, Bo Yuan, Junlan Feng, and Shengyuan Chen. EdgeMem: LLM-free agent memory construction and retrieval via evidence-preserving multi-anchor hypergraph. *arXiv preprint arXiv:2609.05553*, 2026.

[6] Adibvafa Fallahpour, Parsa Idehpour, Vignesh Kothapalli, Nikita Mounier, Shirley Wu, Shayan Pardis, and Jure Leskovec. Memorilla: Latent semantic memory for LLMs. *arXiv preprint arXiv:2610.21375*, 2026.

[7] Mem0. memory-benchmarks. https://github.com/mem0ai/memory-benchmarks; https://mem0.ai/research, 2026.

[8] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL https://arxiv.org/abs/2310.08560. arXiv preprint arXiv:2310.08560.

[9] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

[10] Preston Rasmussen et al. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL https://arxiv.org/abs/2501.13956. arXiv preprint.

[11] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[12] Mohammad Tavakoli, Alireza Salemi, Carrie Ye, Mohamed Abdalla, Hamed Zamani, and J Ross Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs. *arXiv preprint arXiv:2510.27246*, 2025.

[13] Di Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *ICLR 2025*, 2025. URL https://arxiv.org/abs/2410.10813.

[14] Shitao Xiao, Zheng Liu, Peitian Zhang, et al. C-pack: Packed resources for general chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023.

[15] Derong Xu, Yi Wen, Pengyue Jia, Yingyi Zhang, Wenlin Zhang, Yichao Wang, Huifeng Guo, Ruiming Tang, Xiangyu Zhao, Enhong Chen, and Tong Xu. From single to multi-granularity: Toward long-term memory association and selection of conversational agents. *arXiv preprint arXiv:2505.19549*, 2025.

[16] Wujiang Xu et al. A-MEM: Agentic memory for LLM agents. In *NeurIPS 2025*, 2025. URL https://arxiv.org/abs/2502.12110.

A Registered predictions

We registered a numbered prediction before every run except the first 1M run of base retrieval, whose score set the stopping rule for the full run; the predictions sit in a dated ledger file that we release with the code. Development runs use a gpt-5-mini reader and BEAM's judge (gpt-4.1-mini); n is 200 questions per tier unless stated.

Table 1: Registered predictions and outcomes.

Run	Prediction	Measured	Outcome
500K dev, base retrieval	pass 65 to 70	71.5	missed by 1.5, high
500K dev, + planner	+3 to +6 overall; multi-session +10 or more; abstention down by at most 5	+7.7 on the 168 questions both runs answered; multi-session 58.8 to 82.4; abstention 64.7 to 82.4	missed by 1.7, high; held; held
500K dev, + compaction	within 2 of planner alone	−1.8 on the 168 shared questions; abstention 82.4 to 70.6 (two questions); reader tokens 23.7K to 19.2K	held
500K dev, 8-bit vectors, retrieval only	all-gold recall within 2 points of float32	46.1 to 45.7 at 20 turns, 56.1 to 55.8 at 60	held
500K dev, + reader rules, then + session cap	rules +1 to +3 over planner and compaction, abstention down by at most 5; session cap another 0 to +2, multi-session up	on the 168 shared questions: rules +1.2 (76.8 to 78.0), abstention 70.6 to 70.6; cap +1.8 (78.0 to 79.8), multi-session 82.4 to 100.0	held; held; held; held
100K replication, base retrieval and full	base retrieval 72 to 78; full +4 to +8; multi-session +10 or more; abstention down by at most 5	77.0; +2.5; +10.0; −10.0	held; missed, low; held; missed
Both tiers, date rule (final version)	100K temporal 65 or more; 100K overall 80 or more; 500K (200 questions) within 3 of 79.2; 500K temporal (200 questions) within 5 of 73; update and contradiction within 5	60.0; 80.5; 79.0; 65.0; pooled −5.0 and +2.5	missed by one question; held; held; missed by 3.0; held
1M (700 questions), base retrieval, GPT-5, one judge	score unpredicted; the result set the stopping rule for the full run: run it only if base plus half the development gain reached Mem0 + 4 (67.9)	64.7, gap +0.9 [−3.0, +4.4]; forecast 69.3 with the 500K gain, 67.5 with the pooled gain after the temporal fix	rule passed, then 0.4 short after the fix; the full run went ahead
1M (700 questions), full method, GPT-5	one judge 68 or more; BEAM judge 78 or more; multi-session 65 or more; event ordering 45 or more; abstention within 5 of 57.1; cost under $40; store under 1.5 MB per conversation	67.3; 73.9; 70.0; 54.3; 60.0; $34.82 metered plus about $0.16 for 42 questions before a restart; 1.77 MB	missed, low; missed, low; held; held; held; held; missed, high

Run	Prediction	Measured	Outcome
10M, full method, GPT-5, one judge (n=200)	Mem0 40 to 50; Memorable 48 to 60; Memorable ahead by 5 or more; contradiction 40 or more against Mem0 under 20; multi-session within 10 of Mem0; run cost under $20; judge cost under $16 for both files; embedding under 90 minutes; store under 10 MB per conversation	46.5; 56.0; +9.5; 60.0 against 10.0; 25.0 against 30.0; $10.21; $10.45; 35 minutes; 16.7 MB	held; held; held; held; held; held; held; held; missed

B Result tables

Table 2: BEAM-1M pass rate by ability under one judge (Claude Sonnet 5.5, BEAM's rubric prompt), 70 questions per ability. Mem0 is its Platform v3 run with GPT-5; both of ours use GPT-5. Base retrieval is the search path before the planner, compaction and 8-bit vectors are added. Bold marks the best of the three.

Ability	Mem0	Base retrieval	Memorable
Abstention	55.7	57.1	**60.0**
Contradiction resolution	7.1	**65.7**	58.6
Event ordering	51.4	34.3	**54.3**
Information extraction	74.3	78.6	**80.0**
Instruction following	85.7	**87.1**	85.7
Knowledge update	**65.7**	62.9	52.9
Multi-session reasoning	**75.7**	55.7	70.0
Preference following	**97.1**	91.4	94.3
Summarization	**57.1**	50.0	54.3
Temporal reasoning	**68.6**	64.3	62.9
All 700	63.9	64.7	67.3

Table 3: Building and storing the memory of one BEAM-1M conversation (1,710 turns, 4.25M characters). Mem0 OSS 2.0.0 with gpt-5-mini as extractor, measured over 80% of the conversation and scaled.

	Model calls	Tokens in / out	Cost	Time	Store
Memorable	0	0 / 0	$0	about 30 s	1.47 MB
Mem0, measured 80%	683	6.85M / 2.14M	$6.00	260 min	64 MB
Mem0, whole conversation	about 855	8.6M / 2.7M	about $7.50	about 5.4 h	about 80 MB

Table 4: BEAM-10M pass rate by ability, 20 questions per ability over ten conversations. One judge: Claude Sonnet 5.5 on both systems. Own judges: Mem0's GPT-5 judge on its file; BEAM's judge (gpt-4.1-mini) in our run loop.

	One judge		Own judges	
Ability	Mem0	Memorable	Mem0	Memorable
Abstention	40.0	50.0	40.0	85.0
Contradiction resolution	10.0	60.0	25.0	55.0
Event ordering	15.0	20.0	15.0	35.0
Information extraction	55.0	60.0	55.0	60.0
Instruction following	90.0	90.0	90.0	95.0
Knowledge update	80.0	80.0	80.0	80.0
Multi-session reasoning	30.0	25.0	30.0	40.0
Preference following	90.0	75.0	95.0	85.0
Summarization	35.0	50.0	55.0	85.0
Temporal reasoning	20.0	50.0	20.0	45.0
All 200	46.5	56.0	50.5	66.5

C BEAM under each system's own judge

Table 5 gives the pass rates from each system's own judge: BEAM's judge (gpt-4.1-mini under BEAM's rubric prompt) for our runs, and Mem0's own judge (GPT-5) for its published file. Mem0's answers score 70.1 under that judge and 63.9 under ours, which is why the body of the paper uses one judge. We also ran Mem0's published answers through BEAM's judge, which scores them 72.9; under that judge Mem0, base retrieval and Memorable sit at 72.9, 72.1 and 73.9. On contradiction resolution Mem0's answers score 48.6 under its own judge, 15.7 under BEAM's judge and 7.1 under ours. The gap to our 58.6 under the one judge and 42.9 under BEAM's judge holds under both judges that scored both systems, and its size varies with the judge.

Table 5: BEAM-1M pass rate by ability under each system's own judge, with Mem0's answers also scored by BEAM's judge.

Ability	Mem0 own judge	Mem0 BEAM judge	Base retrieval BEAM judge	Memorable BEAM judge
Abstention	55.7	80.0	84.3	88.6
Contradiction resolution	48.6	15.7	64.3	42.9
Event ordering	60.0	74.3	62.9	65.7
Information extraction	75.7	72.9	75.7	74.3
Instruction following	88.6	90.0	88.6	84.3
Knowledge update	65.7	64.3	62.9	60.0
Multi-session reasoning	74.3	80.0	52.9	80.0
Preference following	97.1	98.6	95.7	97.1
Summarization	68.6	91.4	72.9	88.6
Temporal reasoning	67.1	61.4	61.4	57.1
All 700	70.1	72.9	72.1	73.9

The contradiction-resolution drop in the full method, 64.3 to 42.9 under BEAM's judge, we checked question by question: 18 questions base retrieval passed and the full method failed, 3 the other way. Nine of the 18 lost a gold turn from the read set; the other nine kept the gold turns and the reader wrote a dated timeline or resolved to the latest statement and left the conflict unnamed. Only two base-retrieval answers and one full-method answer used the word "contradict" or "conflict" at all. A judge output that lacks a score counts as 0. The judging of our 10M answers logged two such outputs among the 200 questions' rubric items, both on event-ordering questions that fail with or without them; Mem0's 10M file and the 1M files were judged before the script logged them.

D Reader rules and planner prompt

The reader prompt lists the retrieved turns, each as one line with its date and speaker, then the question, then these rules, verbatim:

> Rules: use only the excerpts. Combine and cross-reference every relevant excerpt. If excerpts contradict, prefer the most recent one and mention the change. Keep exact names, numbers and dates. Follow any standing instruction or preference the user gave in the excerpts. For ordering questions list the items in chronological order, one per line. For time questions use the dates shown. For counts, first list every instance with its date, then give the number. Resolve relative times ("two weeks later", "the day before") against the dates shown and state the resulting date. If the user stated a rule or preference more than once, the latest statement wins; ignore rules the user later withdrew. When listing events, merge repeated mentions of the same event. For summaries, cover each phase of the topic in date order. Only when the question itself asks for the current, latest or present value, answer with the most recent value and mention the earlier value as history. For the time between two named events, use the dates given where those events are stated, show both dates and the arithmetic, and note any later change to a date in one short clause. If the excerpts mention the topic but not the specific detail asked for, say that the conversation does not include that detail; never fill in details that are not written in the excerpts.

The planner prompt:

> You plan searches over a long chat history between a user and an assistant. Given the user's question, reply with JSON only: {"type": one of "lookup" (one fact), "aggregate" (combine facts spread over many sessions, counts, lists, comparisons), "order" (sequence or timeline of events), "summary" (summarize a topic), "queries": 2 to 4 short search queries that together cover every part of the question, using the words the conversation would likely use}.

E Compaction rules

Markdown removed: heading marks (`#` at line start), emphasis stars and underscores, horizontal rules, table separator rows and code-fence lines; table pipes become a spaced bar. The filler rule applies to user and assistant turns; the repeat rule to assistant turns only. Filler sentences removed when the whole sentence matches: "Great/Good/Excellent question/point/idea", "Sure/Absolutely/Of course/Certainly", "I hope this helps", "Feel free to ...", "Let me know if ...", "Happy to help", "Thanks for sharing/asking/the update", "Good luck", "Is there anything else ...", "Would you like me to ...?". Repeated assistant sentences: an assistant sentence over 40 characters whose lower-cased, punctuation-stripped form has appeared earlier in the same conversation is dropped; the first occurrence is kept. On the first 1M conversation the most frequent removals, across the filler and repeat rules, were "Certainly!" (388 times) and "If you have any further questions or need additional assistance, feel free to ask!" (127 times).

If the rules empty a turn, its first 200 original characters are kept. The rules have failure cases we know of: the emphasis rule removes every pair of stars, so an expression such as `x**2` loses its operator, and the filler rule matches by sentence shape, so "Let me know if deployment fails at 17:00." is removed although it carries a time. On the first 1M conversation we inspected the removed sentences by frequency: the frequent ones are pleasantries and repeated closings, and we have not read every removed sentence.

F 8-bit vectors against float vectors

We ran the search of Section 2.2 with the planner and the standing pool switched off, on the 269 development questions of the 500K tier that have gold turns, with compaction on, once with bge-small vectors stored as float32 and once as 8-bit integers (Equation 1), and measured how often the gold turns reach the read set. With float32, every gold turn is in the top 20 for 46.1% of questions and in the top 60 for 56.1%, and the share of gold turns found is 56.2% at 20 and 68.0% at 60. With 8-bit vectors the four numbers are 45.7%, 55.8%, 55.8% and 67.7%. One ability moves, by one question each time: information extraction at top 20 falls from 66.7% to 63.3% and at top 60 from 83.3% to 80.0%; the other eight abilities are unchanged at both cutoffs. Reader input at 60 turns is 26,750 tokens with float32 and 26,769 with 8-bit vectors. An earlier

sweep whose settings we did not record gave 48.3% against 48.0% at 20 and 59.1% against 58.7% at 60; we report the rerun, whose per-question rows we release.

G Mem0 build measurement

We ran mem0ai 2.0.0 with its default extraction prompt, gpt-5-mini as the extraction model through an Azure OpenAI endpoint, a local Qdrant vector store and bge-small-en-v1.5 as the embedder. We fed it the first BEAM-1M conversation in Mem0's own batches of two turns and logged every model call. After 683 batches of 855 the metered cost reached $6.00 and we stopped the run: 6,854,553 input and 2,143,893 output tokens, 3,788 memory entries of 242 characters on average (0.92 MB of text, 5.8 MB of float vectors), 64 MB on disk including Qdrant's index and the change-log database, 260 minutes of wall time at one worker. The figures in the body scale these by 855/683. Between 2.0.0 and the current 2.2.1 Mem0's extraction prompt is unchanged and its add path gains only warnings, input checks and one bug fix, so the measurement applies to the current release.